SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 2 **Duration : 1 Hour**
Total Marks:20

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I ***SAHANA S - 192321131*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Sahana S***

Question 1. Hospital Shift Scheduling (Backtracking Search)

Assign doctors D1, D2, D3 to the shifts Morning (M), Afternoon (A) and Night (N), one doctor per shift, subject to the five given constraints.

Step 1: CSP formulation

- **Variables:** D1, D2, D3 (the shift assigned to each doctor).
- **Domain of each variable:** {M, A, N}, ordered M < A < N.
- **Constraints:** (i) D1 ≠ N; (ii) D2 < D3 (D2 works earlier than D3); (iii)/(v) Alldiff(D1, D2, D3), since every doctor gets exactly one shift and every shift exactly one doctor; (iv) D3 ≠ M.

Unary constraints (i) and (iv) shrink the domains before the search starts: D1 ∈ {M, A}, D2 ∈ {M, A, N}, D3 ∈ {A, N}. To show every constraint check, the trace below still tests each value against all constraints. Variable order is D1, D2, D3 and value order is M, A, N.

Step 2: Backtracking search, with every constraint check

#	Partial assignment tried	Constraint checks	Outcome
1	D1 = M	(i) D1 ≠ N ✓; Alldiff ✓	Consistent, go deeper
2	D1 = M, D2 = M	Alldiff ✗ (M already used by D1)	Reject; try next value for D2
3	D1 = M, D2 = A	Alldiff ✓; (ii) cannot be tested yet (D3 unassigned)	Consistent, go deeper
4	D1 = M, D2 = A, D3 = M	(iv) D3 ≠ M ✗; Alldiff ✗	Reject; try next value for D3
5	D1 = M, D2 = A, D3 = A	Alldiff ✗ (A already used by D2)	Reject; try next value for D3
6	D1 = M, D2 = A, D3 = N	(i) ✓; (ii) A < N ✓; (iii) Alldiff ✓; (iv) N ≠ M ✓; (v) all assigned ✓	**All constraints satisfied: GOAL**

Search tree (explored part): root → D1 = M → D2 = M ✗ | D2 = A → D3 = M ✗ | D3 = A ✗ | **D3 = N** ✓. Backtracking was needed three times (steps 2, 4, 5), each time by trying the next value of the most recent variable. The search stops at the first solution.

Remark: the unexplored branch D1 = M, D2 = N would fail because D3 would have to work after Night, which is impossible. Continuing the search after the first solution would also find D1 = A, D2 = M, D3 = N, which is the only other valid schedule.

Step 3: Final valid schedule

Shift	Morning	Afternoon	Night
Doctor	**D1**	**D2**	**D3**

Verification: D1 works Morning, not Night ✓; D2 (Afternoon) works before D3 (Night) ✓; one doctor per shift ✓; D3 is not on Morning ✓; each doctor has exactly one shift ✓.

Question 3. Autonomous Rescue Robot (Online Search in a Partially Observable Environment)

A rescue robot must travel from S to a survivor at G in a grid of rooms with blocked cells, risky zones (extra cost 2), a sensing range of adjacent cells only, and a move cost of 1.

(a) How each constraint affects decision-making

Constraint	Effect on the robot's decisions
Moves: up, down, left, right	Defines the actions and the branching factor (at most 4). Diagonal shortcuts are not allowed, so the Manhattan distance is a lower bound on the remaining cost.
Blocked cells (obstacles)	Make some actions illegal. A plan is valid only if it avoids them, so a plan that crosses an unknown cell can turn out to be invalid later.
Sensing limited to adjacent cells	The environment is partially observable. The robot cannot plan with a complete map, so it must act with partial knowledge, sense after every move and revise its plan. This is the most important constraint, because it makes the problem an *online* problem.
Move cost 1, +2 for a risky zone	Step costs are not uniform (1 or 3), so a plain breadth-first search is not cost-optimal. The planner must compare accumulated path costs, and a longer safe route can be cheaper than a short risky one (a risky detour pays off only if it saves more than 2 steps).
Minimise total cost with safe navigation	The objective is the least total path cost. Safety is handled by never entering a known obstacle and by charging risky cells the extra penalty.
Dynamic knowledge update	The knowledge base (map) must be revised after every observation. Any new fact that invalidates the current plan triggers replanning.

(b) Solution approach: online search agent that replans with Uniform Cost Search

The robot repeats a sense → update → plan → act loop. The planner is Uniform Cost Search (UCS) over the *known* map. Cells not yet seen are assumed free and safe (the *free-space assumption*), which is optimistic and never overestimates the cost. UCS can use the Manhattan-distance heuristic as an A* planner. This is admissible here because every move costs at least 1, and it expands fewer nodes while returning the same optimal plan.

- **Step 1:** Initialise the known map: the grid size, S and G only. Set the current position to S.
- **Step 2 (sense):** Observe the current cell and its four neighbours. Record each as free, blocked or risky.
- **Step 3 (update knowledge):** Merge the new observations into the map. A newly discovered blocked or risky cell may invalidate the current plan.
- **Step 4 (plan):** Run UCS (or A*) from the current cell to G on the known map, with cost 1 per move, +2 for entering a known risky cell, and known obstacles excluded.
- **Step 5 (act):** Execute only the *first* move of the plan, then go back to Step 2.
- **Step 6 (terminate):** Stop when the robot reaches G. If no path exists on the known map, systematically explore the nearest unknown frontier cell (or report the survivor unreachable if the whole reachable region is explored).

Worked example (4×4 grid; the hidden world is unknown to the robot at the start)

	c1	c2	c3	c4
r1	**S**		**#** (hidden obstacle)	
r2				**R** (hidden risky)
r3		**#** (obstacle)		
r4				**G**

Robot at	What it senses	Plan after update (UCS/A*)	Est. cost left	Action, total so far
(1,1)	(1,2) free, (2,1) free	(1,2)→(1,3)→(1,4)→(2,4)→(3,4)→(4,4) (optimistic; unknown cells assumed free)	6	Move to (1,2); total 1
(1,2)	**(1,3) blocked**, (2,2) free	**Replan:** (2,2)→(2,3)→(2,4)→(3,4)→(4,4)	5	Move to (2,2); total 2
(2,2)	(3,2) blocked, (2,3) free	Unchanged: (2,3)→(2,4)→(3,4)→(4,4)	4	Move to (2,3); total 3
(2,3)	**(2,4) risky**, (3,3) free	**Replan:** through (2,4) would cost 3+1+1 = 5, so use (3,3)→(3,4)→(4,4)	3	Move to (3,3); total 4
(3,3)	(3,4) free, (4,3) free	(3,4)→(4,4)	2	Move to (3,4); total 5
(3,4)	(4,4) = G	(4,4)	1	Move to G; **total cost 6**

The robot made two replans (when it discovered the blocked cell and the risky zone), never bumped into an obstacle, never entered the risky zone, and reached G with total cost 6, which equals the optimal cost on the true map.

(c) Intelligent agent, rationality and environment type

Concept	Application to the rescue robot
Agent type	A **goal-based, utility-based, model-based agent**: it maintains an internal map (the model) updated from percepts, plans toward the goal G, and compares plans by path cost.
PEAS: Performance	Reach the survivor, minimise total cost (moves + risk penalties), avoid obstacles and unsafe cells.
PEAS: Environment	A disaster-affected building modelled as a grid of rooms with blocked and risky cells.
PEAS: Actuators	Move Up, Down, Left, Right.
PEAS: Sensors	Short-range sensor giving the status of the four adjacent cells and the current position.
Rationality	The agent chooses, at each step, the action that maximises expected performance (minimum expected total cost) given the percept history and its built-in knowledge. It is rational because it replans whenever new percepts change the best action, and it never takes a known-blocked move.
Environment: observability	**Partially observable** (only adjacent cells are visible).
Environment: agents	**Single agent**.
Environment: determinism	**Deterministic** moves (assuming the robot's actions have the intended effect).
Environment: episodes	**Sequential**: current moves affect later costs and options.
Environment: change	**Static** during planning (blocked cells do not change while it plans). If collapses occurred, it would be dynamic, and the replanning loop would still handle it.
Environment: state space	**Discrete** grid, and **unknown** (the map must be learned online).

(d) Why this approach is the most suitable

- **Handles partial observability:** an offline planner needs the complete map in advance. The online loop works with only local observations and improves its knowledge after every step.
- **Handles non-uniform costs:** UCS orders nodes by accumulated cost g(n), so it gives the cheapest plan when risky cells cost 3 instead of 1. BFS ignores this and is not optimal in cost.
- **Optimal and complete on the known map:** UCS (and A* with an admissible heuristic) always finds the cheapest path, and the optimistic free-space assumption never overestimates. Since the robot does not walk into known obstacles, each replan is safe.

- **Safe and adaptive:** only the first move of each plan is executed and the plan is revised whenever the map changes, so the robot reacts to new obstacles and risky zones immediately.
- **Better than the alternatives:** a purely reactive or hill-climbing agent can oscillate or be trapped in dead ends and does not minimise cost, while exhaustive offline search is impossible without the map. The replanning agent is complete in a safely explorable environment.

Question 6. Cryptarithmetic Puzzle (CSP with Global Constraints)

Solve TWO + TWO = FOUR, where each letter is a different digit (0–9) and the leading letters T and F are not zero.

(i) Variables, domains and constraints

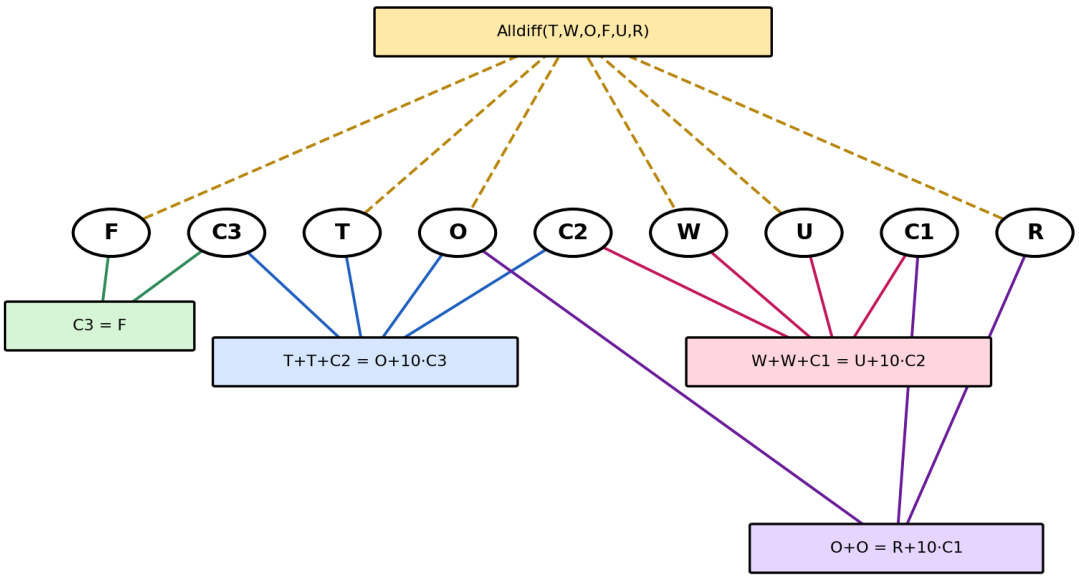
Item	Definition
Variables	T, W, O, F, U, R (letters) and C1, C2, C3 (carries out of the units, tens and hundreds columns)
Domains	T, W, O, F, U, R ∈ {0, 1, …, 9}; C1, C2, C3 ∈ {0, 1}
Constraints	(1) Alldiff(T, W, O, F, U, R); (2) T ≠ 0 and F ≠ 0; (3) the four column equations in (ii)

(ii) Column constraints as equations

Column	Equation
Units	O + O = R + 10·C1
Tens	W + W + C1 = U + 10·C2
Hundreds	T + T + C2 = O + 10·C3
Thousands	C3 = F

(iii) Constraint hypergraph

Circles are variables and boxes are hyperedges (constraints that involve more than two variables). The Alldiff box touches all six letters, and each column constraint touches its own letters and carries.



Circles = variables; boxes = hyperedges (constraints over 2 or more variables)

(iv) Step-by-step backtracking with constraint checks

Variable order: F, T, then O, R, C1, W, U (most constrained first). Reasoning used at each check is shown beside each assignment.

#	Assignment	Constraint check	Result
1	F = 1	C3 = F and C3 ≤ 1, and F ≠ 0, so C3 = 1 (forced)	OK
2	T = 5	T+T+C2 = O+10 gives O = C2 ∈ {0, 1}. O = 1 clashes with F, so O = 0 and C2 = 0. Units: 0+0 = R+10·C1 gives R = 0 = O, violating Alldiff	✗ backtrack

#	Assignment	Constraint check	Result
3	T = 6, C2 = 0	O = 2. Units: 4 = R+10·C1 gives R = 4, C1 = 0. Tens: 2W = U must hold with W ≤ 4 and W, U not in {1, 6, 2, 4}: W = 0 gives U = 0 (= W) ✗; W = 3 gives U = 6 (= T) ✗	✗ backtrack
4	T = 6, C2 = 1	O = 3. Units: 6 = R+10·C1 gives R = 6 = T, violating Alldiff	✗ backtrack
5	T = 7, C2 = 0	O = 4 (14 = 4 + 10). Units: 8 = R+10·C1 gives R = 8, C1 = 0. Tens: W+W = U, so W ≤ 4	OK
6	W = 0	U = 0 = W, violating Alldiff	✗ try next
7	W = 2	U = 4 = O, violating Alldiff	✗ try next
8	W = 3	U = 6. Alldiff on {7, 3, 4, 1, 6, 8} ✓. T ≠ 0 ✓, F ≠ 0 ✓. All three column equations ✓	✓ **solution**

One valid assignment: T = 7, W = 3, O = 4, F = 1, U = 6, R = 8 with carries C1 = 0, C2 = 0, C3 = 1.

		T = 7	W = 3	O = 4
+		T = 7	W = 3	O = 4
=	F = 1	O = 4	U = 6	R = 8

Check: 734 + 734 = 1468 ✓ (units 4+4 = 8; tens 3+3 = 6; hundreds 7+7 = 14, write 4, carry 1; thousands = 1). Note: the puzzle has seven valid assignments in total, and the search above stops at the first. Others include 765 + 765 = 1530 and 928 + 928 = 1856.

Why the constraints help: the global Alldiff and the carry equations let the search reject most branches immediately (F is forced to 1 by C3 before any search; T ≥ 5 is forced by the hundreds column), so only a few assignments are tried.

ii. Show the belief state after each action in the sequence [Right, Right, Right].
iii. Prove that no shorter action sequence guarantees reaching cell 4.
iv. Suppose the robot gains a bump sensor that reports when it tries to move into a wall. Write a contingency plan that reaches cell 4 and compare it with the sensorless plan.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided